



Octave code for the wrapper (1)

- In this lesson, you will learn how to implement the EKF joint state/parameter estimator in Octave, and will see results of applying it to the linear spring-mass-damper model.
 - Note again that even though the model is linear, we still require a nonlinear KF to perform parameter estimation.
 - The unknown parameters are: $\theta = [k, b, m]^T$.
- The Octave wrapper code is the same as for runEKFid except that we call a different “init” function:

```
ekfData = initEKFjoint([xhat0;q0],blkdiag(SigmaX0,SigmaQ0),...
    blkdiag(model.SigmaW,SigmaQW),model.SigmaV,u(1),dT);
```

and a different “iter” function:

```
[Xhatstore(:,k),boundstore(:,k),ekfData] = iterEKFjoint(uk,zk,ekfData);
```



Octave code for initEKFjoint

- The initEKFjoint function stores parameters in ekfData.
- The derivatives are a MESS, so I find them symbolically.

```
function ekfData = initEKFjoint(Xhat0,SigmaX0,SigmaW,SigmaV,priorU,dT)
    ekfData.Xhat = Xhat0;           % Initial state description
    ekfData.SigmaX = SigmaX0;       % Initial estimation-error covariance
    ekfData.SigmaV = SigmaV;        % Sensor-noise covariance
    ekfData.SigmaW = SigmaW;        % Process-noise covariance
    ekfData.priorU = priorU;        % Previous value of input force

    % Compute the derivative of the output with respect to the parameters
    syms k b m x1old x2old uold
    sA = [0 1; -k/m -b/m]; sB = [0; 1/m];
    sAd = expm(sA*dT); sBd = (sA\((sAd-eye(2)))*sB;
    sxold = [x1old; x2old];
    sAhat = simplify(jacobian(sAd*sxold + sBd*uold,[x1old x2old k b m]));

    ekfData.Ad = matlabFunction(sAd); % Ad as a function of parameters
    ekfData.Bd = matlabFunction(sBd); % Bd as a function of parameters
    ekfData.Ahat = matlabFunction(sAhat); % Ahat as a function of stuff
```



Octave code for iterEKFjoint (1)

- The iteration code in iterEKFjoint first unpacks parameters from ekfData, then computes $\hat{A}_{k-1}$:

```
function [Xhat,bounds,ekfData] = iterEKFjoint(uk,zk,ekfData)

    % Get data stored in ekfData structure
    priorU = ekfData.priorU;
    SigmaX = ekfData.SigmaX;
    SigmaV = ekfData.SigmaV;
    SigmaW = ekfData.SigmaW;
    Xhat = ekfData.Xhat;

    % EKF Step 0: Extract components of joint state vector (xhatplus_k-1)
    % and compute Ad, Bd, and Ahat
    x1 = Xhat(1); x2 = Xhat(2); k = Xhat(3); b = Xhat(4); m = Xhat(5);
    Ad = real(ekfData.Ad(b,k,m));
    Bd = real(ekfData.Bd(b,k,m));
    Ahat = ekfData.Ahat(b,k,m,priorU,x1,x2);
    Ahat = [real(Ahat); zeros(3,2), eye(3)];
```



Octave code for iterEKFjoint (2)

- Then, it executes step 1a to 2a.

```
% Step 1a: State estimate time update
Xhat(1:2) = Ad*Xhat(1:2) + Bd*priorU;
% "Q" states don't get updated

% Step 1b: Error covariance time update
%       sigmaminus(k) = Ahat(k-1)*sigmaplus(k-1)*Ahat(k-1)' + ...
%       Bhat(k-1)*sigmawtilde*Bhat(k-1)'
SigmaX = Ahat*SigmaX*Ahat' + SigmaW';

% Step 1c: Output estimate
zhat = Xhat(1);

% Step 2a: Estimator gain matrix
Chat = [1 0 0 0 0];
SigmaZ = Chat*SigmaX*Chat' + SigmaV;
L = SigmaX*Chat'/SigmaZ;
```



Octave code for iterEKFjoint (3)

- It continues with 2b–2c.

```
% Step 2b: State estimate measurement update
r = zk - zhat; % innovation: use to check for sensor errors...
if r^2 > 100*SigmaZ, L(:)=0.0; end
Xhat = Xhat + L*r;

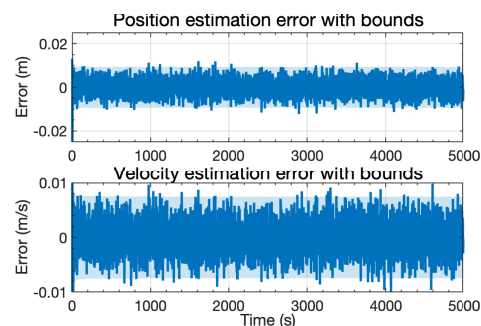
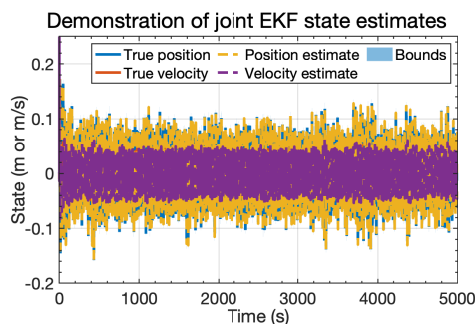
% Step 2c: Error covariance measurement update
SigmaX = SigmaX - L*SigmaZ*L';
[~,S,V] = svd(SigmaX); % Implement Higham method to help robustness
HH = V*S*V';
SigmaX = (SigmaX + SigmaX' + HH + HH')/4;

% Save data in ekfData structure for next time...
ekfData.priorU = uk;
ekfData.SigmaX = SigmaX;
ekfData.Xhat = Xhat;
bounds = 3*sqrt(diag(SigmaX));
end
```



Sample results (joint EKF state estimation)

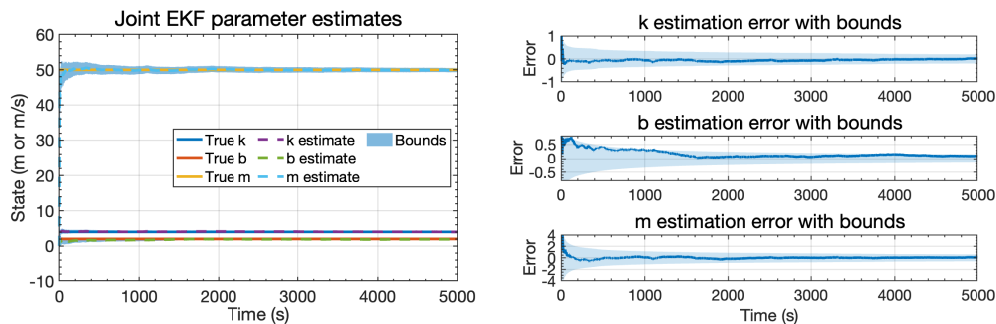
- Some sample state-estimation results from the EKF joint filter are shown below.
- Errors are similar to non-joint state estimation, but perhaps a little larger.





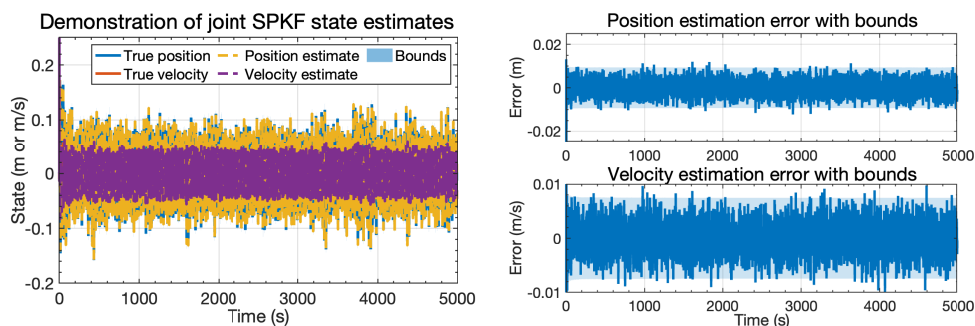
Sample results (joint EKF parameter estimation)

- Some sample parameter-estimation results are shown below.
- Errors are again very similar to non-joint parameter estimation.



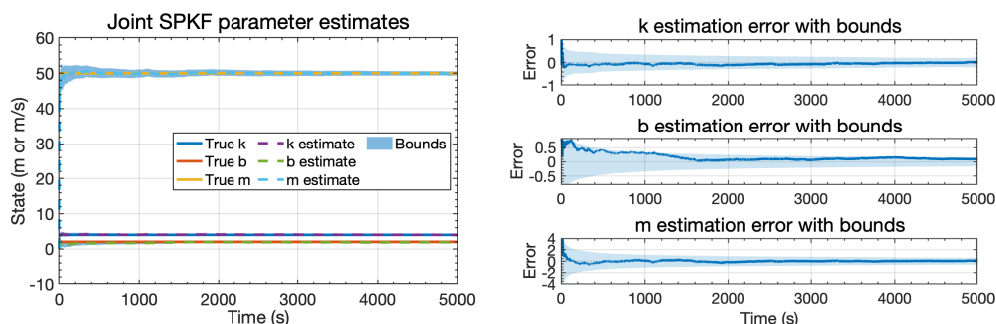
Sample results (joint SPKF state estimation)

- I don't show the code (it is on the Coursera site), but I do share results of joint SPKF state and parameter estimation.
- State-estimation results from the SPKF joint filter are visually indistinguishable from the EKF version.



Sample results (joint SPKF parameter estimation)

- Similarly, parameter-estimation results are nearly impossible to distinguish from their joint-EKF counterparts.
- Which filter you implement is primarily a matter of choice, at least for this relatively simple problem.





Summary

- You have learned how to implement EKF code for joint state and parameter estimation.
- Once again, the code closely follows the steps derived in the previous lesson.
- The biggest challenge with implementing the EKF is finding $\hat{A}_{k-1}$.
 - It is very easy to compute the derivative incorrectly if you are doing it manually.
 - So, I used symbolic methods when I write out the derivative.
- You have seen that results of parameter estimation, when applied to the linear spring-mass-damper model look very good, and very similar to the SPKF results.